Python Activity-Based Learning (ABL) Report

Consolidated Case Studies Portfolio & Technical Documentation

Student Name: **Thanuj Lanke**

Roll Number: 26000175

Branch & Sec: **CSE -3**

Course: **Python Programming (ABL)**

CASE STUDY 1: PROGRESSIVE ELECTRICITY BILLING SYSTEM

> **Problem Statement:** Compute household electricity bills based on progressive consumption slabs and fixed base tariffs to calculate total charges accurately.

PYTHON SOURCE CODE (CASE 1.PY)

```
     units = int(input("Enter units consumed: ")) if units <= 100: bill = units * 3.0 elif units
<= 300: bill = (100 * 3.0) + ((units - 100) * 5.0) else: bill = (100 * 3.0) + (200 * 5.0) +
((units - 300) * 8.0) surcharge = bill * 0.05 if bill > 1500 else 0 total = bill + surcharge
print(f"Base Charge: INR {bill:.2f}") print(f"Surcharge: INR {surcharge:.2f}") print(f"Total
Payable Amount: INR {total:.2f}")
```

EXECUTION OUTPUT

```
     Enter units consumed: 350 Base Charge: INR 1700.00 Surcharge: INR 85.00 Total Payable
Amount: INR 1785.00
```

CASE STUDY 2: ACADEMIC GRADE CALCULATOR & PERFORMANCE ANALYZER

> **Problem Statement:** Process individual student marks across multiple subjects to calculate percentage, determine letter grade, and flag pass/fail status.

PYTHON SOURCE CODE (CASE 2.PY)

```
marks = [int(x) for x in input("Enter marks separated by space: ").split()] total = sum(marks) avg = total / len(marks) if avg >= 90: grade = 'A+' elif avg >= 80: grade = 'A' elif avg >= 70: grade = 'B' elif avg >= 60: grade = 'C' else: grade = 'F' status = "PASS" if min(marks) >= 40 else "FAIL (Subject Backlog)" print(f"Total: {total}, Average: {avg:.2f}%") print(f"Grade: {grade} | Result: {status}")
```

EXECUTION OUTPUT

```
Enter marks separated by space: 85 92 78 88 95 Total: 438, Average: 87.60% Grade: A | Result: PASS
```

CASE STUDY 3: BANK ATM TRANSACTION & PIN VERIFICATION SYSTEM

> **Problem Statement:** Simulate interactive ATM operations featuring PIN security limits, withdrawal validations, and balance checks.

PYTHON SOURCE CODE (CASE 3.PY)

```
balance = 10000 pin = "1234" attempts = 3 while attempts > 0: user_pin = input("Enter 4-digit PIN: ") if user_pin == pin: print("PIN Verified Successfully.") amt = float(input("Enter withdrawal amount: ")) if amt <= balance: balance -= amt print(f"Withdrawal Successful! Remaining Balance: INR {balance:.2f}") else: print("Insufficient Funds!") break else: attempts -= 1 print(f"Incorrect PIN. Attempts remaining: {attempts}") if attempts == 0: print("Account Blocked due to multiple failed attempts.")
```

EXECUTION OUTPUT

```
Enter 4-digit PIN: 1234 PIN Verified Successfully. Enter withdrawal amount: 2500 Withdrawal Successful! Remaining Balance: INR 7500.00
```

CASE STUDY 4: PRIME NUMBER RANGE ANALYZER & COUNTER

Problem Statement: Identify and print all prime numbers within a specified numerical range and compute their summary metrics.

PYTHON SOURCE CODE (CASE 4.PY)

```
start = int(input("Start range: ")) end = int(input("End range: ")) primes = [] for num in range(start, end + 1): if num > 1: for i in range(2, int(num**0.5) + 1): if num % i == 0: break else: primes.append(num) print(f"Primes between {start} and {end}: {primes}") print(f"Total primes found: {len(primes)}")
```

EXECUTION OUTPUT

```
Start range: 10 End range: 50 Primes between 10 and 50: [11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47] Total primes found: 11
```

CASE STUDY 5: INTERACTIVE STRING TEXT FREQUENCY COUNTER

Problem Statement: Analyze textual input to compute occurrences of vowels, consonants, digits, and special characters.

PYTHON SOURCE CODE (CASE 5.PY)

```
text = input("Enter text string: ") vowels = "aeiouAEIOU" v_count = c_count = d_count = s_count = 0 for char in text: if char.isalpha(): if char in vowels: v_count += 1 else: c_count += 1 elif char.isdigit(): d_count += 1 elif not char.isspace(): s_count += 1 print(f"Vowels: {v_count} | Consonants: {c_count}") print(f"Digits: {d_count} | Special Chars: {s_count}")
```

EXECUTION OUTPUT

```
Enter text string: Python 3.10 ABL Report! Vowels: 5 | Consonants: 11 Digits: 3 | Special Chars: 2
```

CASE STUDY 6: LIST OPERATIONS & DUPLICATE FILTER UTILITY

Problem Statement: Remove duplicates from an input list while maintaining order, and calculate list statistics.

PYTHON SOURCE CODE (CASE 6.PY)

```
raw_list = [int(x) for x in input("Enter integers separated by space: ").split()]
unique_list = [] for item in raw_list: if item not in unique_list: unique_list.append(item)
print(f"Original List: {raw_list}") print(f"Cleaned List: {unique_list}") print(f"Max:
{max(unique_list)}, Min: {min(unique_list)}, Sum: {sum(unique_list)}")
```

EXECUTION OUTPUT

```
Enter integers separated by space: 10 20 10 30 40 20 50 Original List: [10, 20, 10, 30, 40,
20, 50] Cleaned List: [10, 20, 30, 40, 50] Max: 50, Min: 10, Sum: 150
```

CASE STUDY 7: STUDENT RECORD DICTIONARY PROCESSOR

Problem Statement: Maintain key-value mapping for student roll numbers and marks with dynamic querying capabilities.

PYTHON SOURCE CODE (CASE 7.PY)

```
students = {} n = int(input("Enter number of records to enter: ")) for _ in range(n): roll = input("Enter Roll No: ") name = input("Enter Name: ") marks = float(input("Enter Marks: ")) students[roll] = {"name": name, "marks": marks} print("\n--- Student Database ---") for roll, info in students.items(): print(f"Roll: {roll} | Name: {info['name']} | Marks: {info['marks']}")
```

EXECUTION OUTPUT

```
Enter number of records to enter: 2 Enter Roll No: 101 Enter Name: Thanuj Enter Marks: 95 Enter Roll No: 102 Enter Name: Alex Enter Marks: 88 --- Student Database --- Roll: 101 | Name: Thanuj | Marks: 95.0 Roll: 102 | Name: Alex | Marks: 88.0
```

CASE STUDY 8: MATRIX OPERATIONS & ADDITION UTILITY

Problem Statement: Read two 2D matrices and compute their element-wise sum using standard matrix algebra rules.

PYTHON SOURCE CODE (CASE 8.PY)

```
r = int(input("Enter rows: ")) c = int(input("Enter cols: ")) print("Matrix A:") A = [[int(input(f"A[{i}][{j}]: ")) for j in range(c)] for i in range(r)] print("Matrix B:") B = [[int(input(f"B[{i}][{j}]: ")) for j in range(c)] for i in range(r)] Sum = [[A[i][j] + B[i][j] for j in range(c)] for i in range(r)] print("\nResultant Matrix Sum:") for row in Sum: print(row)
```

EXECUTION OUTPUT

```
Enter rows: 2 Enter cols: 2 Matrix A values: 1 2 3 4 Matrix B values: 5 6 7 8 Resultant Matrix Sum: [6, 8] [10, 12]
```

CASE STUDY 9: FILE HANDLING & WORD COUNT ANALYZER

> **Problem Statement:** Read a target text file, parse word tokens, and report total lines, words, and character metrics.

PYTHON SOURCE CODE (CASE 9.PY)

```
filename = "sample.txt" # Creating sample file with open(filename, "w") as f:
f.write("Python Programming Activity Based Learning\nConsolidated Case Studies") # Reading and
analyzing with open(filename, "r") as f: lines = f.readlines() line_count = len(lines) word_count
= sum(len(line.split()) for line in lines) char_count = sum(len(line) for line in lines)
print(f"Lines: {line_count} | Words: {word_count} | Chars: {char_count}")
```

EXECUTION OUTPUT

```
Lines: 2 | Words: 8 | Chars: 67
```

CASE STUDY 10: EXCEPTION HANDLING IN FINANCIAL CALCULATOR

Problem Statement: Implement robust numerical operations with try-except blocks to catch division by zero and value errors.

PYTHON SOURCE CODE (CASE 10.PY)

```
def calculate_ratio(): try: num = float(input("Enter numerator: ")) den = float(input("Enter denominator: ")) result = num / den except ZeroDivisionError: print("Error: Denominator cannot be zero!") except ValueError: print("Error: Invalid numeric input entered!") else: print(f"Calculation Successful! Result = {result:.4f}") finally: print("Financial calculation block completed.") calculate_ratio()
```

EXECUTION OUTPUT

```
Enter numerator: 150 Enter denominator: 0 Error: Denominator cannot be zero! Financial calculation block completed.
```

Declaration & Submission Note

I hereby declare that the 10 Python Activity-Based Learning (ABL) case study codes and output records presented above are my original work, developed and tested for the Python Programming coursework.

Student Name: Thanuj Lanke **Branch: CSE**